

R3.L01 — Représentations et codages des images

TP n°2 & TD n°2 : Classe ColorImage, Format PPM & Échantillonnage Bilinéaire

*Guide intégral d'explications et d'architectures basé sur le cours de M. **Éric Rémy***

Explications détaillées du POURQUOI, schémas vectoriels, corrigé TD2 et code C++ complet

Contrôle TD : 14/10/2026

Contrôle TP : 06/11/2026 (ou 21/10)

Séance TP2 : ColorImage & Bilinéaire

Le Modèle Couleur RGB & La Classe Color

Définition de la classe Color

CM Diapo 38 / 87

```
class Color {  
public:  
    uint8_t r, g, b; // Données membres publiques (Slide 38)  
  
    inline Color(uint8_t _r = 0, uint8_t _g = 0, uint8_t _b = 0)  
        : r(_r), g(_g), b(_b) {}  
  
    friend bool operator==(const Color& c1, const Color& c2) {  
        return c1.r == c2.r && c1.g == c2.g && c1.b == c2.b;  
    }  
    friend Color operator*(double alpha, const Color& c) {  
        return Color((uint8_t)(alpha * c.r),  
                     (uint8_t)(alpha * c.g),  
                     (uint8_t)(alpha * c.b));  
    }  
    friend Color operator+(const Color& c1, const Color& c2) {  
        return Color(c1.r + c2.r, c1.g + c2.g, c1.b + c2.b);  
    }  
};
```

Le POURQUOI : Structure Mémoire & Opérateurs

- **Pas de padding mémoire** : Un `uint8_t` a un alignement de 1. La structure `Color` occupe **exactement 3 octets en RAM** sans aucun trou d'alignement : `sizeof(Color) == 3`.
- **Pourquoi surcharger `*` et `+` ?** M. Rémy le précise textuellement : « Les opérateurs `*` et `+` seront nécessaires au ré-échantillonnage bilinéaire » pour écrire naturellement des moyennes pondérées de pixels.

r (1 octet) g (1 octet) b (1 octet)

Attributs et Représentation 1D continue

CM Diapo 39 / 87

- **Attributs privés :**
`const uint16_t width, height;` (dimensions immuables)
`Color* array;` (tableau 1D de `width * height` éléments)
- **Calcul d'index 2D → 1D :**
`pixel(x, y) = array[y * width + x]`
Ordonnancement Row-Major : les pixels d'une même ligne sont consécutifs en RAM.
- **Accesseurs sécurisés :** vérification `x >= width || y >= height` avec jet d'une exception `std::out_of_range`.

Le POURQUOI : Compatibilité Binaire Directe avec PPM

Puisque `sizeof(Color) == 3` (r, g, b consécutifs), le tableau `array` est en mémoire une suite continue d'octets :

```
R0 G0 B0 | R1 G1 B1 | R2 G2 B2 | ... | Rn Gn Bn
```

- **Écriture/Lecture en 1 seule instruction système :**
`os.write((const char*)array, width * height * 3);`
Aucun recalcul, aucune boucle, débit disque maximal !
- **Pas de constructeur par défaut ni assignation :**
`ColorImage() = delete;` et `operator= = delete;` évitent les images orphelines et les copies superficielles accidentelles.

Le Format PPM : P6 (Binaire) vs P3 (ASCII)

Caractéristique	PPM Binaire (P6) — Recommandé	PPM Texte (P3) — Bonus
Magic Number	"P6"	"P3"
En-tête Netpbm	P6 # commentaire <largeur> <hauteur> 255	
Codage des pixels	Octets bruts (Raw bits) : 3 octets par pixel (R, G, B de 0 à 255)	Texte décimal ASCII : valeurs séparées par des espaces ou
Taille fichier	Compact : taille = entête + (W × H × 3) octets	Volumineux : 3 à 5 fois plus gros que le P6
Lecture C++	<code>is.read((char*)array, W * H * 3);</code>	Boucle <code>is >> r >> g >> b;</code> pour chaque pixel

Le POURQUOI : Pourquoi la norme PPM exige P6 et 255 ?

- 1. Performance** : Le format P3 impose de convertir chaque entier texte en binaire via `std::istream::operator>>`, ce qui est extrêmement lent sur des images de plusieurs millions de pixels.
- 2. Standard 24 bits TrueColor** : La dynamique fixée à 255 correspond rigoureusement à 1 octet par canal primaire ($256 \text{ nuances} \times 256 \times 256 = 16,7 \text{ millions de couleurs}$).

Lecture et Écriture PPM (writePPM & readPPM)

Méthode writePPM(ostream&) const

TP2.txt Lignes 15-20

```
void ColorImage::writePPM(std::ostream& os) const {  
    os << "P6\n# Image sauvegardee pour RepCod.\n"  
    << width << " " << height << "\n255\n";  
    os.write((const char*)array, width * height * 3);  
}
```

- On écrit width * height * 3 octets (3 octets par pixel).
- Pas de saut de ligne final dans les pixels : les données sont brutes !

Le POURQUOI : Le Piège de l'octet séparateur après 255

Après lecture de la dynamique `is >> dyn`, le flux s'arrête **immédiatement avant** le caractère d'espacement (généralement).

```
char sep; is.get(sep); // Consomme EXACTEMENT 1 octet blanc !  
is.read((char*)res->array, w * h * 3);
```

- **Pourquoi ne PAS faire `is >> std::ws` ?** `std::ws` consomme tous les espaces consécutifs. Si le premier pixel commence par la valeur 0x20 (espace) ou 0x0A (), il sera détruit !
- La norme Netpbm spécifie « *strictly one whitespace character* » après maxval : un simple `is.get(sep)` respecte rigoureusement la norme.

Pourquoi rééchantillonner une image ?

- **Changement d'échelle (Scaling)** : Adapter une image source de taille $W \times H$ à une grille de destination $w \times h$ (agrandissement ou réduction).
- **Rapport d'aspect (Aspect Ratio)** : Si $W/H \neq w/h$, l'image sera anamorphosée (ex: passage de 80×60 [4:3] à 823×400 [~2:1], aplatissement visible).
- **Théorème de Shannon-Nyquist** : Pour restituer fidèlement un détail, la fréquence d'échantillonnage doit être au moins le double de la fréquence maximale du signal.

Le POURQUOI : Projection Inverse (Destination vers Source)

- **Projection directe (Source → Destination)** : Si on parcourt l'image source, lors d'un agrandissement plusieurs pixels d'arrivée ne recevront aucun point (création de **trous noirs** indésirables).
- **Règle d'or de l'imagerie** : On parcourt **toujours l'image de destination** pixel par pixel (x', y') , et on vient chercher sa couleur dans l'image source à (x, y) . Chaque pixel d'arrivée est ainsi garanti d'être calculé !

Rééchantillonnage Simpliste : simpleScale(...)

Méthode ColorImage::simpleScale (Plus Proche Voisin)

TP2.txt &
TD3

```
ColorImage* ColorImage::simpleScale(uint16_t w, uint16_t h) const {
    if (w == 0 || h == 0)
        throw std::invalid_argument("Dimensions non nulles");
    ColorImage* res = new ColorImage(w, h);
    for (uint16_t y_prime = 0; y_prime < h; ++y_prime) {
        uint16_t y = (uint32_t(y_prime) * height) / h;
        for (uint16_t x_prime = 0; x_prime < w; ++x_prime) {
            uint16_t x = (uint32_t(x_prime) * width) / w;
            res->pixel(x_prime, y_prime) = pixel(x, y);
        }
    }
    return res;
}
```

Le POURQUOI : Principe et Défauts Visuels

- **Formule mathématique** : Pour le point (x', y') d'arrivée, on tronque à l'entier inférieur :

$$x = \lfloor (x' \times W) / w \rfloor \quad y = \lfloor (y' \times H) / h \rfloor$$

- **Attention au débordement arithmétique** : $x_prime * width$ peut dépasser 65535 sur 16 bits ! Le cast explicite en `uint32_t` avant la multiplication évite l'overflow.
- **Le défaut majeur : L'effet de blocs (Pixellisation)**. Chaque pixel source se transforme en gros pavé uniforme. L'aliasing spatial est maximal (marches d'escalier très nettes).

Principe Mathématique de l'Interpolation Biliéaire

Les 4 Voisins et les Poids dx, dy

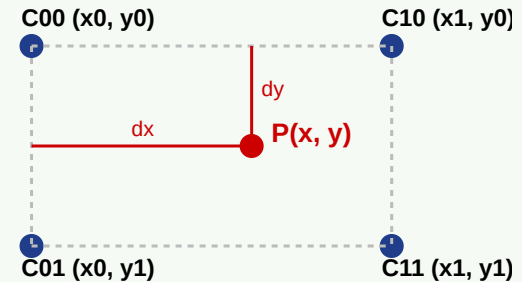
CM Diapo 38 & TP2.txt

Au lieu de tronquer, on conserve la coordonnée réelle continue :

$$\mathbf{x} = (\mathbf{x}' \times \mathbf{W}) / \mathbf{w}, \quad \mathbf{y} = (\mathbf{y}' \times \mathbf{H}) / \mathbf{h}$$

- Partie entière : $\mathbf{x}_0 = \lfloor \mathbf{x} \rfloor$, $\mathbf{y}_0 = \lfloor \mathbf{y} \rfloor$. Voisins : $\mathbf{x}_1 = \mathbf{x}_0 + 1$, $\mathbf{y}_1 = \mathbf{y}_0 + 1$.
- Partie fractionnaire : $\mathbf{dx} = \mathbf{x} - \mathbf{x}_0 \in [0; 1[$, $\mathbf{dy} = \mathbf{y} - \mathbf{y}_0 \in [0; 1[$.
- Interpolation horizontale :
 $C_{\text{haut}} = (1 - \mathbf{dx}) \times C_{00} + \mathbf{dx} \times C_{10}$
 $C_{\text{bas}} = (1 - \mathbf{dx}) \times C_{01} + \mathbf{dx} \times C_{11}$
- Interpolation verticale finale :
 $C = (1 - \mathbf{dy}) \times C_{\text{haut}} + \mathbf{dy} \times C_{\text{bas}}$

Schéma de la grille d'interpolation



Plus le point P est proche d'un sommet, plus ce sommet pèse lourd dans la couleur finale.

Implantation de bilinearScale(...) en C++

Code complet de ColorImage::bilinearScale

TP2.txt Question Rééchantillonnage bilinéaire

```
ColorImage* ColorImage::bilinearScale(uint16_t w, uint16_t h) const {
    if (w == 0 || h == 0) throw std::invalid_argument("Dimensions strictement positives");
    ColorImage* res = new ColorImage(w, h);
    for (uint16_t y_prime = 0; y_prime < h; ++y_prime) {
        double y = (double(y_prime) * height) / h;
        uint16_t y0 = uint16_t(y);
        uint16_t y1 = (y0 + 1 < height) ? y0 + 1 : y0; // Clamping bord bas pour éviter out_of_range
        double dy = y - y0;

        for (uint16_t x_prime = 0; x_prime < w; ++x_prime) {
            double x = (double(x_prime) * width) / w;
            uint16_t x0 = uint16_t(x);
            uint16_t x1 = (x0 + 1 < width) ? x0 + 1 : x0; // Clamping bord droit
            double dx = x - x0;

            const Color& c00 = pixel(x0, y0); const Color& c10 = pixel(x1, y0);
            const Color& c01 = pixel(x0, y1); const Color& c11 = pixel(x1, y1);

            // Utilisation directe des opérateurs * et + surchargés pour Color !
            Color c_top = (1.0 - dx) * c00 + dx * c10;
            Color c_bot = (1.0 - dx) * c01 + dx * c11;
            res->pixel(x_prime, y_prime) = (1.0 - dy) * c_top + dy * c_bot;
        }
    }
    return res;
}
```

Le POURQUOI : Clamping des bords et Opérateurs Color

Si $x_0 == \text{width} - 1$, le point suivant x_1 déborderait hors du tableau ! L'instruction conditionnelle $x_1 = (x_0 + 1 < \text{width}) ? x_0 + 1 : x_0$; évite tout crash mémoire sur le bord droit et inférieur.

Les opérateurs surchargés $*$ et $+$ permettent une écriture pure et élégante, strictement identique à la formule mathématique.

Le Décalage Demi-Pixel (-0.5 / +0.5) (TP2.txt)

Pourquoi les deux images sautent-elles ?

TP2.txt Lignes 47-48

- En alternant `tp2_simple.ppm` et `tp2_bilinear.ppm`, on observe un **décalage visible** du chat vers le haut-gauche.
- **Origine physique** : Un pixel n'est pas un point abstrait, mais une petite case carrée de largeur 1 ! Son **centre optique** est à $(x' + 0.5, y' + 0.5)$.
- Lorsqu'on multiplie par $823 / 80 \approx 10$, la demi-largeur de 0.5 est multipliée par 10 → le décalage fait **5 pixels entiers** à l'écran !

La Correction Mathématique de M. Rémy

Pour aligner parfaitement les centres des pixels :

1. On se place au centre du pixel d'arrivée : $x' + 0.5$.
2. On applique le facteur d'échelle : $((x' + 0.5) \times W) / w$.
3. On repasse en coordonnées d'indice du pixel source en retranchant le demi-pixel : $- 0.5$.

```
double x = ((x_prime + 0.5) * width) / w - 0.5;  
if (x < 0.0) x = 0.0; // Protection aux bords
```

« Si vous choisissez de ne pas le faire, vous aurez quand même pratiquement tous les points attachés à cette question. » (M. Rémy).

Endianness & Patron swap_bytes (TD2 Exo 2)

Patron de fonction générique swap_bytes

TD2 Exo 2 & corrigé_prof.txt

```
template <typename T>
void swap_bytes(T& val) {
    int8_t* ptr = reinterpret_cast<int8_t*>(&val);
    size_t n = sizeof(T);
    for (size_t i = 0; i < n / 2; ++i) {
        int8_t tmp = *(ptr + i);
        *(ptr + i) = *(ptr + n - i - 1);
        *(ptr + n - i - 1) = tmp;
    }
}
```

Détection & Portabilité

```
inline bool is_little_endian() {
    uint16_t test = 0x0001;
    return *((char*)&test) == 1;
}

template <typename T>
void from_big_endian(T& val) {
    if (is_little_endian()) swap_bytes(val);
}
```

Le Piège d'Examen : Pourquoi ne pas swapper les chaînes ?

- **Entier 16 bits & Double 64 bits IEEE 754** : Ce sont des nombres scalaires atomiques répartis sur plusieurs octets. Si on ne les permute pas de Big à Little, leur valeur est détruite → **swap_bytes obligatoire !**
- **FourCC (char[4]) & Chaîne de texte** : C'est une suite de caractères indépendants d'1 octet chacun (sizeof(char) == 1). Si le fichier contient "JPEG", swapper produirait "GEPJ", ce qui est une faute grave ! → **JAMAIS de swap_bytes sur du texte !**

Tailles des Imageurs en RAM

TD2 Exercice 3.1

- **Smartphone 24 bits (4032 × 3024) :**
 $12\,192\,768 \text{ px} \times 3 \text{ o} = \mathbf{36,58 \text{ Mo}}$ (34,88 Mio).
Objet sur PC 32 bits = 8 o (ptr 4 o) vs PC 64 bits = 16 o (ptr 8 o + padding).
Négligeable !
- **Reflex numérique 24 bits (7360 × 4912) :**
 $36\,152\,320 \text{ px} \times 3 \text{ o} = \mathbf{108,46 \text{ Mo}}$ (103,43 Mio).
- **Scanner 3D (512³ voxels sur 16 bits) :**
 $512^3 = 2^{27} \text{ voxels} \times 2 \text{ o} = 2^{28} \text{ o} = \mathbf{256 \text{ Mio}}$ (268,44 Mo).

Débit Brut 4K 60 fps & Nécessité de la Compression

$$\text{Débit} = 3840 \times 2160 \times 24 \text{ bits} \times 60 \text{ fps} = \mathbf{11,94 \text{ Gbit/s}}$$

Comparé aux débits réels du monde numérique :

- **ADSL (20 Mbit/s) :** Flux 4K brut **600 fois supérieur !**
- **TNT (25 Mbit/s) :** Flux 4K brut **478 fois supérieur !**
- **Fibre FTTH 100M :** Flux 4K brut **120 fois supérieur !**
- **5G théorique (2,1 Gbit/s) :** Flux brut encore **5,7 fois supérieur !**

Conclusion du cours : Transmettre de la vidéo sans compression est matériellement impossible. Les codecs (H.264, H.265, AV1) doivent compresser d'un facteur 100 à 500 !

Synthèse pour le Contrôle Théorique & Pratique

Questions types du Contrôle Théorique (14/10)

- **P6 vs P5** : P6 écrit des triplets RGB de 3 octets par pixel ($W \times H \times 3$) tandis que P5 écrit 1 seul octet par pixel ($W \times H$).
- **Séparateur après 255** : Un seul octet blanc selon la norme Netpbm. Utiliser `is.get ( sep )` pour ne pas avaler le premier pixel.
- **Endianness** : Ne s'applique qu'aux nombres atomiques > 1 octet. Ne jamais swapper de chaînes ASCII.

Points clés du Contrôle Pratique TP (06/11)

- **Parcours inverse obligatoire** : On boucle toujours sur (x', y') de l'image de destination pour éviter les trous lors d'un agrandissement.
- **Gestion rigoureuse de la mémoire** : Tout `new ColorImage(...)` renvoyé par `simpleScale` ou `bilinearScale` doit avoir son `delete` dans le `main` !
- **Prototypes stricts** : Respecter à la lettre la casse et les paramètres du sujet (noté par script automatique).

Annexe Code (1/5) : ColorImage.hpp (Intégral)

Fichier d'en-tête ColorImage.hpp (Classes Color & ColorImage)

Header C++

```
#ifndef COLORIMAGE_HPP
#define COLORIMAGE_HPP
#include <stdint>
#include <iostream>
#include <stdexcept>
#include <string>

class Color {
public:
    uint8_t r, g, b;
    inline Color(uint8_t _r = 0, uint8_t _g = 0, uint8_t _b = 0) : r(_r), g(_g), b(_b) {}
    friend bool operator==(const Color& c1, const Color& c2) { return c1.r == c2.r && c1.g == c2.g && c1.b == c2.b; }
    friend Color operator*(double a, const Color& c) { return Color((uint8_t)(a * c.r), (uint8_t)(a * c.g), (uint8_t)(a * c.b)); }
    friend Color operator+(const Color& c1, const Color& c2) { return Color(c1.r + c2.r, c1.g + c2.g, c1.b + c2.b); }
};

class ColorImage {
private:
    const uint16_t width, height;
    Color* array;
    ColorImage() = delete;
public:
    ColorImage& operator=(const ColorImage& b) = delete;
    ColorImage(uint16_t w, uint16_t h);
    ColorImage(const ColorImage& orig);
    ~ColorImage();
    inline const uint16_t& getWidth() const { return width; }
    inline const uint16_t& getHeight() const { return height; }
    inline Color& pixel(uint16_t x, uint16_t y) {
        if (x >= width || y >= height) throw std::out_of_range("Hors limites");
        return array[y * width + x];
    }
    inline const Color& pixel(uint16_t x, uint16_t y) const {
        if (x >= width || y >= height) throw std::out_of_range("Hors limites");
        return array[y * width + x];
    }
    void clear(Color c = Color());
    void rectangle(uint16_t x, uint16_t y, uint16_t w, uint16_t h, Color c);
    void fillRectangle(uint16_t x, uint16_t y, uint16_t w, uint16_t h, Color c);
    void writePPM(std::ostream& os) const;
    static void skip_line(std::istream& is);
    static void skip_comments(std::istream& is);
    static ColorImage* readPPM(std::istream& is);
};
```

Annexe Code (2/5) : ColorImage.cpp (Base & Formes)

ColorImage.cpp : Constructeurs, Destructeur, clear, rectangles

Partie 1/3

```
#include "ColorImage.hpp"

ColorImage::ColorImage(uint16_t w, uint16_t h) : width(w), height(h), array(nullptr) {
    if (w == 0 || h == 0) throw std::invalid_argument("Dimensions > 0");
    array = new Color[width * height];
}

ColorImage::ColorImage(const ColorImage& o) : width(o.width), height(o.height), array(nullptr) {
    array = new Color[o.width * o.height];
    for (size_t t = 0; t < size_t(width * height); t++) array[t] = o.array[t];
}

ColorImage::~ColorImage() { delete[] array; }

void ColorImage::clear(Color c) {
    for (size_t t = 0; t < size_t(width * height); t++) array[t] = c;
}

void ColorImage::rectangle(uint16_t x, uint16_t y, uint16_t w, uint16_t h, Color c) {
    if (w == 0 || h == 0) return;
    if (x + w > width || y + h > height) throw std::out_of_range("Rectangle hors limites");
    for (uint16_t i = 0; i < w; ++i) { pixel(x + i, y) = c; pixel(x + i, y + h - 1) = c; }
    for (uint16_t j = 0; j < h; ++j) { pixel(x, y + j) = c; pixel(x + w - 1, y + j) = c; }
}

void ColorImage::fillRectangle(uint16_t x, uint16_t y, uint16_t w, uint16_t h, Color c) {
    if (w == 0 || h == 0) return;
    if (x + w > width || y + h > height) throw std::out_of_range("Rectangle plein hors limites");
    for (uint16_t j = 0; j < h; ++j)
        for (uint16_t i = 0; i < w; ++i) pixel(x + i, y + j) = c;
}
```

Annexe Code (3/5) : ColorImage.cpp (Entrées / Sorties PPM)

ColorImage.cpp : writePPM, skip_comments, readPPM (P6 & P3)

Partie 2/3

```
void ColorImage::writePPM(std::ostream& os) const {
    os << "P6\n# Image sauvegardee pour RepCod.\n" << width << " " << height << "\n255\n";
    os.write((const char*)array, width * height * 3);
}

void ColorImage::skip_line(std::istream& is) { char c; while (is.get(c) && c != '\n') {} }

void ColorImage::skip_comments(std::istream& is) {
    char c;
    while (is.get(c)) {
        if (c == ' ' || c == '\t' || c == '\n' || c == '\r') continue;
        if (c == '#') skip_line(is); else { is.putback(c); break; }
    }
}

ColorImage* ColorImage::readPPM(std::istream& is) {
    std::string magic = ""; if (!(is >> magic) || (magic != "P6" && magic != "P3")) throw std::runtime_error("Format non PPM");
    skip_comments(is);
    uint16_t w = 0, h = 0; if (!(is >> w >> h)) throw std::runtime_error("Dimensions absentes");
    skip_comments(is);
    int dyn = 0; if (!(is >> dyn) || dyn != 255) throw std::runtime_error("Dynamique != 255");
    ColorImage* res = new ColorImage(w, h);
    if (magic == "P6") {
        char sep; is.get(sep); // Consomme exactement un caractere blanc apres 255
        is.read((char*)res->array, w * h * 3);
    } else { // Lecture P3 ASCII
        for (size_t t = 0; t < size_t(w * h); ++t) {
            int r, g, b; is >> r >> g >> b; res->array[t] = Color((uint8_t)r, (uint8_t)g, (uint8_t)b);
        }
    }
    return res;
}
```

ColorImage.cpp : Les deux méthodes de rééchantillonnage spatial

Partie 3/3

```
ColorImage* ColorImage::simpleScale(uint16_t w, uint16_t h) const {
    if (w == 0 || h == 0) throw std::invalid_argument("Dimensions strictement positives");
    ColorImage* res = new ColorImage(w, h);
    for (uint16_t y_prime = 0; y_prime < h; ++y_prime) {
        uint16_t y = (uint32_t(y_prime) * height) / h;
        for (uint16_t x_prime = 0; x_prime < w; ++x_prime) {
            uint16_t x = (uint32_t(x_prime) * width) / w;
            res->pixel(x_prime, y_prime) = pixel(x, y);
        }
    }
    return res;
}

ColorImage* ColorImage::bilinearScale(uint16_t w, uint16_t h) const {
    if (w == 0 || h == 0) throw std::invalid_argument("Dimensions strictement positives");
    ColorImage* res = new ColorImage(w, h);
    for (uint16_t y_prime = 0; y_prime < h; ++y_prime) {
        double y = (double(y_prime) * height) / h;
        uint16_t y0 = uint16_t(y);
        uint16_t y1 = (y0 + 1 < height) ? y0 + 1 : y0;
        double dy = y - y0;

        for (uint16_t x_prime = 0; x_prime < w; ++x_prime) {
            double x = (double(x_prime) * width) / w;
            uint16_t x0 = uint16_t(x);
            uint16_t x1 = (x0 + 1 < width) ? x0 + 1 : x0;
            double dx = x - x0;

            const Color& c00 = pixel(x0, y0);
            const Color& c10 = pixel(x1, y0);
            const Color& c01 = pixel(x0, y1);
            const Color& c11 = pixel(x1, y1);

            Color c_top = (1.0 - dx) * c00 + dx * c10;
            Color c_bot = (1.0 - dx) * c01 + dx * c11;
            res->pixel(x_prime, y_prime) = (1.0 - dy) * c_top + dy * c_bot;
        }
    }
    return res;
}
```

tp2.cpp : Scénario complet demandé par TP2.txt

Programme Principal

```
#include "ColorImage.hpp"
#include <iostream>
#include <fstream>

int main() {
    try {
        // 1. Test lecture/ecriture et formes sur chat.ppm
        std::ifstream fChat("chat.ppm", std::ios::binary);
        if (!fChat) throw std::runtime_error("chat.ppm introuvable");
        ColorImage* chat = ColorImage::readPPM(fChat);
        chat->rectangle(20, 20, 80, 50, Color(255, 0, 0)); // Cadre rouge
        chat->fillRectangle(150, 30, 60, 60, Color(0, 255, 0)); // Carre vert
        std::ofstream fChatOut("tp2_chat.ppm", std::ios::binary);
        chat->writePPM(fChatOut);
        delete chat;
        std::cout << "-> tp2_chat.ppm genere avec succes !" << std::endl;

        // 2. Test Reechantillonnage de chat_petit.ppm (80x60 -> 823x400)
        std::ifstream fPetit("chat_petit.ppm", std::ios::binary);
        if (!fPetit) throw std::runtime_error("chat_petit.ppm introuvable");
        ColorImage* petit = ColorImage::readPPM(fPetit);

        // Agrandissement simpliste (Nearest Neighbor)
        ColorImage* simple = petit->simpleScale(823, 400);
        std::ofstream fSimple("tp2_simple.ppm", std::ios::binary);
        simple->writePPM(fSimple);
        delete simple;
        std::cout << "-> tp2_simple.ppm genere (823x400)" << std::endl;

        // Agrandissement bilineaire (Bilinear Interpolation)
        ColorImage* bilinear = petit->bilinearScale(823, 400);
        std::ofstream fBilinear("tp2_bilinear.ppm", std::ios::binary);
        bilinear->writePPM(fBilinear);
        delete bilinear;
        delete petit;
        std::cout << "-> tp2_bilinear.ppm genere (823x400)" << std::endl;
        std::cout << "TOUS LES TESTS DU TP2 SONT VALIDES !" << std::endl;
    } catch (const std::exception& e) {
        std::cerr << "ERREUR : " << e.what() << std::endl; return 1;
    }
    return 0;
}
```